

First In, First Out

Lecture Notes — Week 5: Queues, Circular Buffers, Deques, and Priority

Auqib Hamid Lone

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

September 17, 2026

5.1 Why This Week Exists: Arrival Order Needs Its Own Contract

Read alongside: Horowitz & Sahni, *Fundamentals of Data Structures*, Ch. 3 (chapter-level); Aho, Hopcroft & Ullman, *Data Structures and Algorithms*, Ch. 2 (chapter-level).

Four questions organize this chapter, stated here exactly as the deck states them: why does a stack ruin a print queue; how does one index calculation reclaim wasted array slots; what can a deque do that a queue cannot — and what does it cost; and where does the most urgent job wait when everything is an array. Compressed to a plan: name the queue contract, fix its array with circular arithmetic, open both ends, then rank by priority, and close with the Unit-I review.

The handoff from Week 4 is exact. Week 4 gave us conversion: infix to postfix with one left-to-right stack pass, infix to prefix by reverse-swap-postfix-reverse, and prefix to postfix right-to-left (Labs P3, P4) — all $O(n)$ worst case. It priced recursion: Hanoi needs $2^n - 1 = \Theta(2^n)$ moves but only $O(n)$ live frames on the runtime stack. What it never did: hold items in *arrival* order. Every Week 4 structure is last-in, first-out. This week’s job gives the calculator a waiting room: jobs leave in the order they arrived, the buffer never walks off the end of its array, urgent jobs jump the line — honestly priced (Labs P5, P6).

The calculator pipeline makes the need concrete. Weeks 3–4 built an expression processor: tokenize, check brackets, convert, evaluate — one program learning new tricks each week. The new deficiency: two users hit “print” at once. The second job must *wait its turn* — a stack would serve the newcomer first. The same need appears everywhere: keyboard buffers, process scheduling, server request queues, and (Week 12) the BFS frontier. The course design principle fires again here: every structure in this course answers a deficiency the previous one just exhibited. The stack’s deficiency is its exit order — the queue is the answer. One evolving program, not twelve disconnected topics.

Two arc notes close the framing. Week 2 priced things: name the operation, give the bound, state the case — and every claim in this chapter repeats that shape. Week 4’s close promised: “what changes when the first element out is the first one in — circular buffers, deques, and priority.” Week 6 re-implements this week’s contract on linked lists; Week 7 compares the two; Week 12 spends a whole traversal inside a queue. The arc again: state the contract, implement it once, analyse what it costs.

5.2 The Queue Contract and Its Linear Array

Read alongside: Karumanchi, *Data Structures and Algorithms Made Easy*, Ch. 5, pp.205–223 (PDF) — the Queue ADT and its array implementation; Sedgewick & Wayne, *Algorithms*, Sec. 1.3, p. 120 — queue as a collection ADT.

5.2.1 Why a stack ruins a print queue

Jobs arrive: Alice, Bob, Carol. Fairness means Alice prints first — she arrived first. Push all three on a stack and pop: Carol prints first. The last arrival jumps the line; Alice starves while newcomers keep coming. **Wrong exit order is not a bug you can patch** — LIFO cannot be tuned into FIFO. The contract itself must change. The requirement in one line: insert at one end, remove at the other — first in, first out.

Definition (Queue). A *queue* is a collection with one removal rule: remove the least recently inserted item. *enqueue* adds at the *rear*; *dequeue* removes from the *front*; *front* peeks without removing. Empty dequeue is an error (the queue exception). (Karumanchi Ch. 5, pp.205–223 PDF; Sedgewick Sec. 1.3, p. 120) [1, 2].

Sedgewick’s framing is worth keeping verbatim: bag, queue, and stack differ *only* in which item leaves next — the same contract shape as Week 3’s stack, with the removal rule swapped. Two new symbols today: *front* (next to dequeue) and *rear* (last enqueued) — both array indices, 0-based, per the course Notation Registry.

5.2.2 Linear array: the first attempt

On $Q[0..n-1]$ with capacity n :

```
enqueue(x): rear = rear + 1; Q[rear] = x
dequeue(): x = Q[front]; front = front + 1; return x
empty when front > rear; start with front = 0, rear = -1
```

Each operation touches one slot: $O(1)$ worst case — the same habit as the stack’s $O(1)$ push/pop. But *front* only marches right, and the slots it leaves behind are the subject of the next two subsections.

Figure 5.1 draws the state the deck draws: five slots, X already dequeued and its slot freed, live items A, B, C at indices 1–3, index 4 empty, *front* = 1, *rear* = 3. The oldest item leaves at *front*; the newest waits at *rear*.

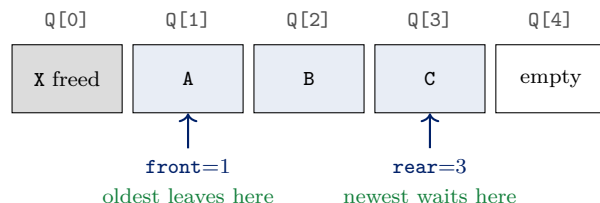


Figure 5.1: Linear queue: X was dequeued and its slot sits empty while *front* marches away from it. The queue holds 3 items but owns 5 slots.

Example 5.2.1 (Worked trace: enqueue, enqueue, dequeue). Start with *front* = 0, *rear* = -1, empty. *enqueue(A)*: *rear* becomes 0, $Q[0] = A$; state [A], *front* = 0, *rear* = 0. *enqueue(B)*: *rear* becomes 1; state [A B], *front* = 0, *rear* = 1. *enqueue(C)*: *rear* becomes 2; state [A B

$C]$, $front = 0$, $rear = 2$. *dequeue* returns $Q[front]$ before the increment — that ordering is load-bearing, because off-by-one here returns the wrong job — so it returns A and $front$ becomes 1; state $[B\ C]$, $front = 1$, $rear = 2$. *enqueue*(D): $rear$ becomes 3; state $[B\ C\ D]$, $front = 1$, $rear = 3$. After five operations the live items sit at indices 1–3; index 0 is dead space the implementation cannot reuse.

5.2.3 The false-full trap

Keep dequeuing and enqueueing: $front$ and $rear$ both drift right until $rear = n - 1$ with live items fewer than n . The implementation reports **full** — but slots 0 to $front - 1$ stand empty. Overflow with free space: a false alarm. The naive fix (shift everything left on each dequeue) restores honesty at $O(n)$ worst case per dequeue — paying linear time to defend a constant-time contract. The real fix, which is the next section: let the indices wrap around the end of the array instead of marching off it — one modular addition, no shifting, $O(1)$ kept.

Example 5.2.2 (Prediction check: capacity 4, enqueue A, B, C, D , then dequeue twice, then enqueue E). Of A : index 0 (dead slot reused); B : index 4 (overflow!); C : index 2 — the answer the first-attempt code actually produces is B . Trace $front/rear$ as numbers, not pictures. After four enqueues $front = 0$, $rear = 3$. Two dequeues move $front$ to 2 (returning A , then B). *enqueue*(E) sets $rear = 3 + 1 = 4$: past the end of a capacity-4 array. A claims the linear code looks left — it never does, so it cannot reuse index 0. C mistakes the live count (3 items) for a slot number — E does not take “the third slot,” it takes $rear + 1$.

5.3 Circular Queue: One Index Calculation Reclaims the Array

Read alongside: Karumanchi, Ch. 5, pp.205–223 (PDF) — wraparound arithmetic, the full/empty tests, and the array cost analysis.

5.3.1 Wraparound arithmetic

Definition (Circular advance (the rule)). Advance every index modulo the capacity: $rear = (rear + 1) \bmod n$, $front = (front + 1) \bmod n$. Index $n - 1$ is followed by index 0 — the array is used as a ring. (Karumanchi Ch. 5, pp.205–223 PDF) [1].

Concrete instance, exactly as the deck states it: capacity 8, $rear = 7 \rightarrow$ next $rear = (7 + 1) \bmod 8 = 0$. The “past the end” slot is the freed slot at the start. Cost of the fix: one division remainder per operation — still $O(1)$ worst case. No shifting, ever.

Figure 5.2 draws the ring the deck draws: capacity 8, five live items B – F at indices 1–5, $front = 1$, $rear = 5$, slots 6, 7, and 0 standing empty, slot 0 already recycled once, with the dashed wraparound arc marking the $7 \rightarrow 0$ step. The next *enqueue* lands at index 6.

5.3.2 Full or empty? They look identical

With wraparound, $front = rear$ after the first enqueue *and* when the ring is completely full — one equality, two meanings. Two honest fixes, both stated exactly as the deck states them. **Fix A (count):** keep count of live items; empty when $count = 0$, full when $count = n$ — one extra integer. **Fix B (sacrificed slot):** declare full when $(rear + 1) \bmod n = front$; the ring holds at most $n - 1$ items and one slot always stands empty. Course convention (Labs P5): use the count version — it holds all n items and its tests read plainly. Name your choice on paper — examiners check the test, not the fashion.

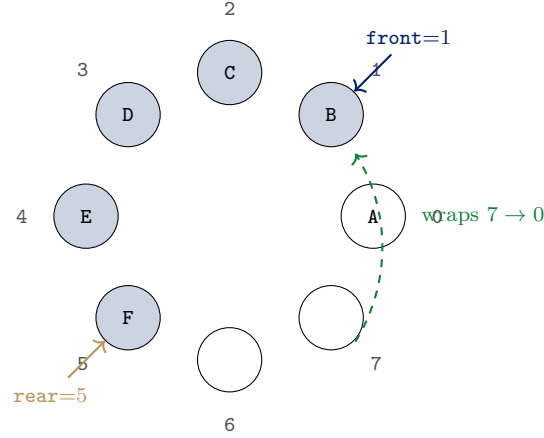


Figure 5.2: Circular queue, capacity 8: five live items (B–F), $\text{front} = 1$, $\text{rear} = 5$; the next `enqueue` lands at index 6, and slot 0 was already recycled once.

Example 5.3.1 (Worked trace: wrapping around (capacity 4, count version)). *enq A, enq B, enq C: $\text{front} = 0$, $\text{rear} = 2$, $\text{count} = 3$. $\text{deq} \rightarrow A$, $\text{deq} \rightarrow B$: each dequeue advances front modularly ($0 \rightarrow 1 \rightarrow 2$) and decrements the count; now $\text{front} = 2$, $\text{rear} = 2$, $\text{count} = 1$ (only C alive). enq D: $\text{rear} = (2 + 1) \bmod 4 = 3$; $\text{front} = 2$, $\text{rear} = 3$, $\text{count} = 2$. enq E: $\text{rear} = (3 + 1) \bmod 4 = 0$ — the first wrap; $\text{front} = 2$, $\text{rear} = 0$, $\text{count} = 3$. enq F: $\text{rear} = (0 + 1) \bmod 4 = 1$ — the second wrap; $\text{front} = 2$, $\text{rear} = 1$, $\text{count} = 4 = \text{full}$. Watch rear walk $2 \rightarrow 3 \rightarrow 0 \rightarrow 1$: the wrap steps are where the linear code died, and the modular code does not even pause. The full test fires honestly: $\text{count} = 4 = n$ with every slot genuinely occupied.*

Cost and code shape (Lab P5), in Week 2’s shape: `enqueue`, `dequeue`, `front` — each one array access plus one mod — $O(1)$ worst case, $O(1)$ auxiliary space beyond the n -slot array ($n = \text{capacity}$ here). The ring is the week’s load-bearing idea: the same wraparound serves keyboard buffers, streaming windows, and Week 12’s BFS frontier. Lab P5 implements exactly this trace: circular `enqueue` / `dequeue` / `display` with the full and empty tests above. The habit: name the operation (index advance), give the bound ($O(1)$), state the case (worst). “Circular is $O(1)$ ” without the case is half an answer.

5.4 Double-Ended Queue: Both Ends Open

Read alongside: Karumanchi, Ch. 5, pp.205–223 (PDF) — the deque and its restricted variants (standard treatment for the naming).

Definition (Deque). A *deque* (“deck”) allows *enqueue* and *dequeue* at both ends: `enqueueFront`, `enqueueRear`, `dequeueFront`, `dequeueRear`. A queue is a deque with two of the four operations switched off. (Karumanchi Ch. 5, pp.205–223 PDF; standard treatment) [1].

Motivation, exactly as the deck states it: an undo history takes new states at the rear but drops the *oldest* states from the front when full — one structure, both ends working. Array version: a circular ring with `front` allowed to step *backwards*: $\text{front} = (\text{front} - 1 + n) \bmod n$. The ring wraps underneath exactly as in the previous section; the new freedom is that `front` steps either direction, and so does `rear`.

Example 5.4.1 (Worked deque state and its restricted cousins). *Start empty (capacity 5): `enqueueRear(Q)`, `enqueueRear(R) \rightarrow rear` grows right; `enqueueFront(P) \rightarrow front` steps left with wraparound. The*

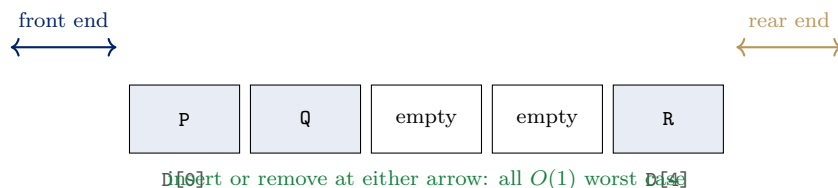


Figure 5.3: Deque strip: insert or remove at either arrow, all $O(1)$ worst case. The ring wraps underneath as in Fig. 5.2; the new freedom is bidirectional stepping.

state drawn in Fig. 5.3 is $[P \ Q \ _ \ _ \ R]$ with both arrows live. Two restricted cousins, named exactly as the deck names them. **Input-restricted deque**: insert at one end only, remove at either — models a priority lane that admits newcomers singly. **Output-restricted deque**: insert at either end, remove at one end only — models a pipeline with a single outlet. Exam note, verbatim: if a question says “insert at rear, delete at both ends”: that is the input-restricted deque. Name it, then trace it — the name earns the first mark, the trace the rest.

5.5 Priority Queue Using Arrays: Urgency Instead of Arrival

Read alongside: Karumanchi, Ch. 7, pp.369–408 (PDF) — the Priority Queue ADT and its array designs; Sedgewick & Wayne, Sec. 2.4, p. 308 — priority queues.

When arrival order is the wrong order: the OS scheduler holds processes (`job`, `priority`) — a system interrupt (priority 0) must run before a background backup (priority 9), however late it arrived. FIFO would run the backup first. Hospital triage, printer “urgent” flags, Dijkstra’s frontier (Week 12): every one ranks by **priority**, not by arrival time. Syllabus constraint, stated plainly: implement it *using arrays* — no heaps yet (heaps arrive with trees in Week 11). Two honest array designs, opposite bills.

Definition (Priority queue). A **priority queue** returns the minimum (or maximum) priority item first. Insert is **enqueue** with a priority; removal is **deleteMin** (or **deleteMax**). (Karumanchi Ch. 7, pp.369–408 PDF; Sedgewick Sec. 2.4, p. 308) [1, 2].

design	insert	deleteMin
Unordered (append)	$O(1)$ worst case	scan all n : $O(n)$ worst
Ordered (sorted)	shift room: $O(n)$ worst	read off end: $O(1)$ worst

No free lunch with arrays: one end of the trade is always linear. (Week 11’s heap makes both $O(\log n)$ — the reason heaps exist.) Choose by workload: many inserts, few removals $\rightarrow$ unordered; removals dominate $\rightarrow$ ordered.

Example 5.5.1 (Trace `deleteMin` on the drawn queues). *Unordered row holds $J:4$, $K:7$, $L:9$, $M:1$, $N:5$. Scan all five priorities, return $M:1$, close the gap by shifting (unordered) — that scan is the $O(n)$ worst-case bill. Ordered row holds the same five jobs sorted ($L:9$, $K:7$, $N:5$, $J:4$, $M:1$); take the end slot — the $O(1)$ worst-case read. Lab P6 implements the unordered version. The price was paid at insertion instead: keeping the row sorted cost a shift per insert ($O(n)$ worst case).*

Example 5.5.2 (Prediction check: 1 000 inserts then 3 removals, or 3 inserts then 1 000 removals?). *Of A: ordered for both; B: unordered, then ordered; C: unordered for both — the answer is B. Price*

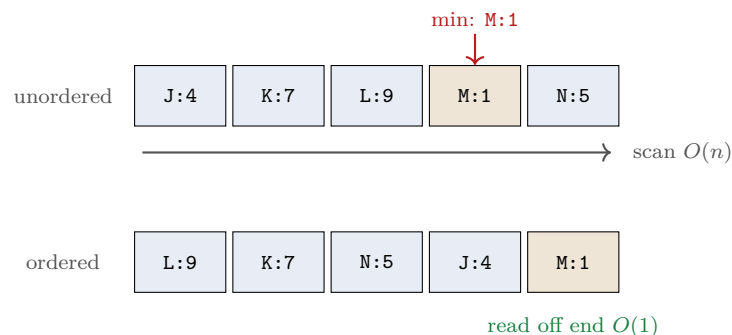


Figure 5.4: Priority queue on arrays: unordered row scanned at $O(n)$ worst case to find M:1; ordered row read off the end at $O(1)$ worst case.

the workload, not the structure: count which operation runs hot, then read the table. Many inserts with few removals: pay the linear bill on the rare operation (the scan) and keep the frequent one constant — unordered. Removals dominant: pay at insertion and keep each removal constant — ordered. A sorts once up front even when removals are rare (paying $O(n)$ per insert to save three scans); C scans a thousand times when one sort would have served.

5.6 Unit-I Map, Mistakes, Summary, and What Comes Next

Read alongside: Weeks 1–4 decks for the traced tables; no new anchor reading in this section.

Unit I in one map (Class Test 1), reproduced exactly as the deck tabulates it:

week	contract	load-bearing skill
1	ADT; <code>malloc/free</code> , NULL	ownership tracing
2	$T(n)$, $O/\Omega/\Theta$, best/worst/average	price every claim
3	Stack ADT; <code>top</code> = -1 empty; $O(1)$	bracket check, postfix eval
4	Notations; conversion; recursion; Hanoi $\Theta(2^n)$	stack traces
5	Queue; <code>front/rear</code> ; mod ring	buffer traces (today)

The test rewards the course habit: name the structure, trace it line by line, state operation + bound + case. Every structure so far lives in an array; Week 6 asks what pointers buy us instead.

Three common mistakes across all of Unit I, stated exactly as the deck states them. **Confusing the case with the bound** — “best case $O(n)$ ” mixes the input (case) with the growth class (bound). Say “best case $\Theta(1)$, worst case $\Theta(n)$ ” with the operation named. **Testing full as `front` = `rear`** — that test also fires on empty. Use `count` (0 vs. n) or sacrifice a slot, and write the test down before the code. **Forgetting direction** — stacks pop the young end, queues the old end, dequeues either end, priority queues the urgent end. State the removal rule first; the code follows it. How to revise: re-trace one table per week (Weeks 3, 4, and today’s two) with a covered answer column — the test is traces, not definitions.

In summary: **Queue** — enqueue at `rear`, dequeue from `front`; $O(1)$ worst case each; the linear array dies of false fullness. **Circular queue** — $(\cdot + 1) \bmod n$ recycling; full vs. empty by `count` or sacrificed slot; Lab P5. **Deque** — four operations, both ends, backward-stepping `front`; restricted cousins name which end is closed. **Priority queue on arrays** — unordered ($O(1)$ insert, $O(n)$ `deleteMin`) vs. ordered (flipped); Lab P6; heaps in Week 11. Next week: linked lists — what changes when nodes carry their own links instead of sitting in numbered slots — and the queue reborn on pointers (Week 7).

Example 5.6.1 (Before-you-go predictions (Class Test 1 warm-up), with the deck's answers). Answer from the algorithms, then check in the labs (P5, P6). (1) Capacity 5, **front** = 3, **rear** = 1, **count** = 4: full or not? After one **dequeue** and one **enqueue**, what are the three values? Not full ($4 < 5$); then **front** = 4, **rear** = 2, **count** = 4 — the dequeue advances **front** $(3 + 1) \bmod 5 = 4$ with count 3, and the enqueue advances **rear** $(1 + 1) \bmod 5 = 2$ with count back to 4. (2) Name the deque variant: insert rear-only, remove either end. Input-restricted deque. (3) Unordered priority queue holds A:3, B:1, C:2: **deleteMin** returns what, and what does it cost worst case? Returns B:1 at $O(n)$ worst case — the scan touches all three.

Exercises

Answer from the algorithms, then check in the labs: circular traces are Lab P5, the unordered priority queue is Lab P6.

1. Linear queue, capacity 5, **front** = 0, **rear** = -1: run **enqueue(A)**, **enqueue(B)**, **dequeue**, **enqueue(C)**, **enqueue(D)**, **dequeue**. Give **front**, **rear**, and the live contents after each step, and name the dead slot(s).
2. Circular queue (count version), capacity 4: start from **front** = 0, **rear** = 3, **count** = 4 (full). Run **dequeue**, **dequeue**, **enqueue(X)**, **enqueue(Y)**, **enqueue(Z)**. Give the three values after each step and show both wraparound computations.
3. Capacity 6 (count version), **front** = 5, **rear** = 4, **count** = 6: is it full? After two dequeues and one enqueue, what are **front**, **rear**, **count**? (Genuinely new instance reusing the deck's modular rule.)
4. Deque, capacity 5: **enqueueRear(A)**, **enqueueRear(B)**, **enqueueFront(C)**, **dequeueRear**, **enqueueFront(D)**. Give the live contents and state which restricted variant (if any) this operation sequence stays inside.
5. Unordered priority queue holds P:5, Q:2, R:4: **deleteMin** returns what at what worst-case cost? If instead 500 inserts precede 2 removals, which array design does the deck's workload rule pick, and why?

Solutions

1. **front** = 2, **rear** = 3, live [C D]; dead slot: index 0 (and index 1 after the second dequeue). Trace: start (0, -1, empty). **enq A** $\rightarrow$ (0, 0, [A]). **enq B** $\rightarrow$ (0, 1, [A B]). **deq** $\rightarrow$ A, (1, 1, [B]); index 0 dead. **enq C** $\rightarrow$ (1, 2, [B C]). **enq D** $\rightarrow$ (1, 3, [B C D]). **deq** $\rightarrow$ B, (2, 3, [C D]); indices 0–1 dead and unrecoverable by the linear code.
2. **Ends at front = 2, rear = 2, count = 4 (full again); wraps at the enqueues of X (3 $\rightarrow$ 0) and Y (0 $\rightarrow$ 1); Z refused.** Trace: start (0, 3, 4). **deq** $\rightarrow$ (1, 3, 3). **deq** $\rightarrow$ (2, 3, 2). **enq X**: **rear** = $(3 + 1) \bmod 4 = 0 \rightarrow$ (2, 0, 3). **enq Y**: **rear** = $(0 + 1) \bmod 4 = 1 \rightarrow$ (2, 1, 4). **enq Z**: refused — **count** = 4 = n , full; the honest test blocks the write.
3. **Full (6 = n). After the three operations: front = 1, rear = 5, count = 5.** Full test: **count** = 6 = n . Dequeue: **front** = $(5 + 1) \bmod 6 = 0$, **count** 5. Dequeue: **front** = $(0 + 1) \bmod 6 = 1$, **count** 4. Enqueue: **rear** = $(4 + 1) \bmod 6 = 5$, **count** 5. Not full ($5 < 6$).
4. **Live [D C A] (front to rear); the sequence is a full deque — it uses both-end insertion, so it stays inside neither restricted variant.** Trace: **enqRear(A)** $\rightarrow$ [A]; **enqRear(B)** $\rightarrow$ [A B]; **enqFront(C)** $\rightarrow$ [C A B]; **deqRear** removes B $\rightarrow$ [C A]; **enqFront(D)** $\rightarrow$ [D C A]. Input-restricted allows insert at one end only and output-restricted allows remove at one end only — this sequence inserts at both ends (**enqRear** and **enqFront**), so it needs the unrestricted deque.

5. **Returns Q:2 at $O(n)$ worst case; 500 inserts with 2 removals $\rightarrow$ unordered.** `deleteMin` scans all three priorities and returns the minimum Q:2; the scan is $O(n)$ worst case. Workload rule (deck's table): pay the linear bill on the rare operation — with inserts hot and removals rare, keep insert $O(1)$ (unordered) and accept the $O(n)$ scan twice.

References

- [1] Narasimha Karumanchi. *Data Structures And Algorithms Made Easy*. CareerMonk Publications, 2017. This calibre-generated edition carries no printed folio numbers; page numbers cited in CS301 material are PDF page indices. NOT a source for Week 1's C-specific pointer/malloc content — see this course's supporting-books index for the coverage check.
- [2] Robert Sedgewick and Kevin Wayne. *Algorithms*. Addison-Wesley, 4th edition, 2011.